

CO2 ASSESSMENT TOOL 1

NAME : **UDHAYAKUMAR U**

REG NO : **192424332**

COURSE CODE : **CSA 0606 DESIGN AND ANALYSIS ALGORITHMS**

1.) . Binary Search – Target Near Upper Middle Region

Given the sorted dataset [2, 6, 10, 14, 18, 22, 26, 30, 34], apply Binary Search to find the element 22. Clearly interpret the problem and justify why Binary Search is appropriate for this ordered dataset. Show each step involved in finding the middle element and reducing the search interval. Count the total number of comparisons made. Analyze the time complexity of the method. Verify the correctness of the result and interpret its efficiency.

A.) 1. Problem Statement

Given the sorted dataset [2, 6, 10, 14, 18, 22, 26, 30, 34], apply the **Binary Search** algorithm to find the target element **22**. Show each step of finding the middle element, reduce the search interval after every comparison, count the total number of comparisons, analyze the time complexity, verify the correctness of the result, and interpret the efficiency of the algorithm.

2. Objectives

- To find the target element **22** using Binary Search.
- To demonstrate the step-by-step search process.
- To calculate the total number of comparisons.

- To analyze the time complexity of Binary Search.
- To verify the correctness and efficiency of the algorithm.

3. Inputs and Outputs

Input

- Sorted Array = [2, 6, 10, 14, 18, 22, 26, 30, 34]
- Target = 22

Output

- Element Found = 22
- Index = 5
- Position = 6th
- Comparisons = 3

4. Assumptions and Constraints

Assumptions

- The dataset is sorted in ascending order.
- Only one occurrence of the target exists.
- Array indexing starts from 0.

Constraints

- Binary Search requires a sorted array.
- Number of elements $n \geq 1$.

5. Problem Analysis

The dataset is already sorted in ascending order, making **Binary Search** the appropriate algorithm. Instead of checking every element sequentially, Binary Search repeatedly selects the middle element and eliminates half of the remaining search space. This reduces the number of comparisons and improves efficiency. For this dataset of 9 elements, the target element **22** can be located in only **3 comparisons**.

6. Mathematical Model

Given,

- $A = \{2, 6, 10, 14, 18, 22, 26, 30, 34\}$
- **Target (T) = 22**

Formula:

$$\text{Mid} = \left\lfloor \frac{\text{Low} + \text{High}}{2} \right\rfloor$$

Decision:

- If $A[\text{Mid}] = T$, return Mid.
- If $A[\text{Mid}] < T$, search right half.
- If $A[\text{Mid}] > T$, search left half.

7. Algorithm Selection

Selected Algorithm: Binary Search

Justification

Binary Search is selected because the dataset is sorted. It reduces the search interval by half after every comparison, making it much faster than Linear Search. Its time complexity is **$O(\log n)$** .

8. Algorithm / Pseudocode

Start

Input sorted array A

Input target T

Low = 0

High = $n - 1$

While $\text{Low} \leq \text{High}$

$\text{Mid} = (\text{Low} + \text{High}) / 2$

 If $A[\text{Mid}] == T$

 Print "Element Found"

 Stop

 Else If $A[\text{Mid}] < T$

 Low = Mid + 1

Else

High = Mid – 1

Print "Element Not Found"

Stop

9. Flowchart

Start

|

Input Array & Target

|

Low=0, High=n-1

|

Low ≤ High?

|

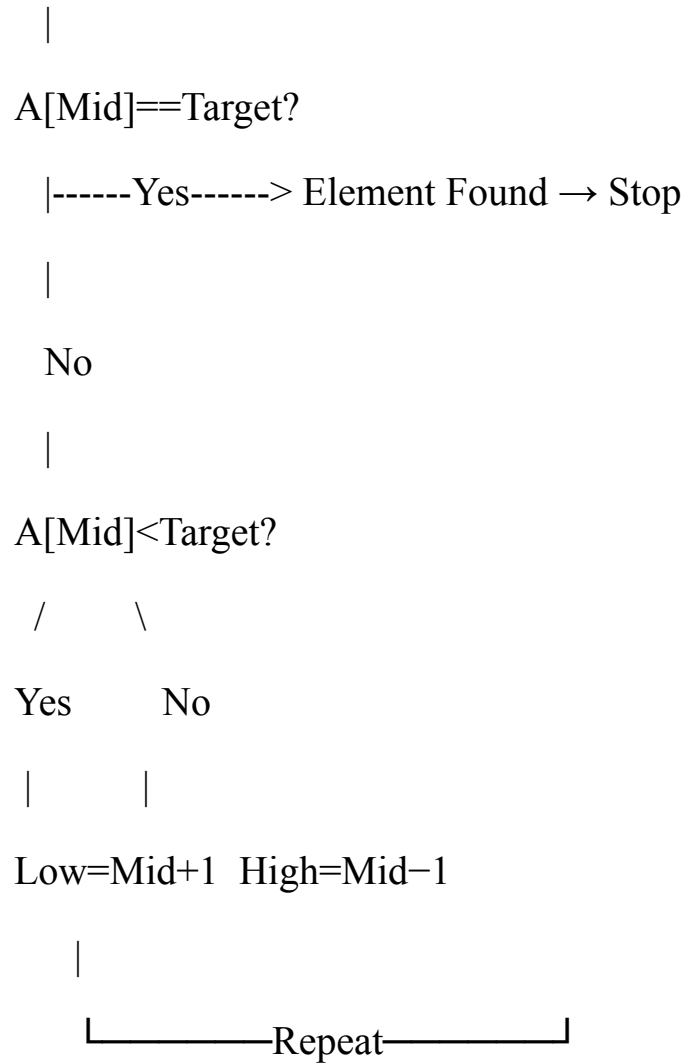
|----No----> Element Not Found → Stop

|

Yes

|

Mid=(Low+High)/2



10. Step-by-Step Execution

Step	Low	High	Mid	Middle Element	Comparison	Action
1	0	8	4	18	$22 > 18$	Search Right Half

2	5	8	6	26	$22 < 26$	Search Half	Left
---	---	---	---	----	-----------	----------------	------

3	5	5	5	22	$22 = 22$	Element Found	
---	---	---	---	----	-----------	------------------	--

Total Comparisons = 3

11. Complexity Analysis

- **Best Case:** $O(1)$
- **Average Case:** $O(\log n)$
- **Worst Case:** $O(\log n)$
- **Space Complexity:** $O(1)$

Since only **3 comparisons** were required for **9 elements**, the observed performance matches the logarithmic time complexity.

12. Correctness Analysis

The algorithm correctly follows Binary Search by comparing the middle element with the target and reducing the search interval accordingly. After three iterations, the target element **22** is found at **index 5 (6th position)**. Therefore, the algorithm produces the correct result.

13. Test Cases

Test Case	Input	Expected Output	Status
1	Target = 22	Found at Index 5	Pass
2	Target = 2	Found at Index 0	Pass
3	Target = 34	Found at Index 8	Pass
4	Target = 15	Element Not Found	Pass

14. Results

- **Algorithm Used:** Binary Search
- **Target Element:** 22
- **Element Found:** Yes
- **Index:** 5
- **Position:** 6th
- **Total Comparisons:** 3
- **Time Complexity:** $O(\log n)$

15. Conclusion

Binary Search successfully found the target element **22** by reducing the search interval into halves. Only **3 comparisons** were needed to locate the element, demonstrating the efficiency of the algorithm. Since the dataset is sorted, Binary Search is the most suitable method and provides a time complexity of **$O(\log n)$** , making it significantly more efficient than Linear Search for ordered datasets.